

Knowledge Transfer-Based Co-Evolutionary Search for Energy-Saving Distributed Heterogeneous Flexible Job Shop Scheduling

Tao Li , Xingchen Li , Junna Zhang , *Member, IEEE*, Dong Liu , and Zidong Wang , *Fellow, IEEE*

Abstract—This paper investigates the energy-saving distributed heterogeneous flexible job shop scheduling problem (ES-DHFJSP), with the dual objectives of reducing both makespan and total energy consumption. To address ES-DHFJSP, a knowledge transfer-based co-evolutionary search algorithm (KTCE) is proposed. Three key components are incorporated into the algorithm. First, a greedy population initialization strategy is designed, by which the characteristics of multi-factory distributed systems are leveraged to generate superior initial individuals. Second, a dynamic neighborhood search structure centered on critical factory is developed, through which the algorithm's capability to explore promising regions around elite solutions is enhanced. Third, a knowledge transfer mechanism centered on a factory allocation probability model is implemented, wherein the optimal factory allocations from previous generations are utilized to construct the probability model and the evolution of subsequent population is guided. The algorithm's convergence rate is substantially improved by this mechanism. The effectiveness and efficiency of KTCE are validated through comprehensive experimental results on various benchmark instances, where superior performance is demonstrated in solving ES-DHFJSP.

Index Terms—Evolutionary algorithm, knowledge transfer, Co-evolutionary search, distributed heterogeneous flexible job shop scheduling, energy-conserving.

I. INTRODUCTION

The rapid advancement of the global economy has led to an unprecedented surge in demand for high-quality products [1]. Traditional single-factory production systems have proven inadequate in meeting these demands, particularly in

terms of rapid, large-scale production of high-quality goods [2]. Distributed manufacturing systems, characterized by multiple factories collaboratively processing product sets, have emerged as a promising solution by significantly enhancing production throughput. This approach offers notable advantages, including enhanced operational flexibility and reduced management risks [3]. Nevertheless, conventional distributed manufacturing systems continue to face critical challenges, primarily manifesting as suboptimal production efficiency and excessive energy consumption [4]. These limitations pose substantial challenges to existing job shop scheduling methodologies [5]. Consequently, there is an urgent need to develop advanced job scheduling techniques that can simultaneously optimize production efficiency and minimize energy consumption.

At present, energy-saving distributed flexible job shop scheduling (ES-DFJSP) presents an effective solution to aforementioned challenges. This method requires solving three fundamental sub-problems: 1) determining optimal job-to-factory allocation schemes; 2) assigning operations to specific machines; and 3) establishing operation processing orders within each factory. The interdependence of these three subproblems makes the ES-DFJSP highly complex, characterized by a vast solution space and significant computational difficulty. Therefore, ES-DFJSP is classified as an NP-hard problem [6].

Evolutionary algorithms possess remarkable advantages in addressing multi-objective optimization challenges and have achieved notable results. Among them, noteworthy examples include the differential evolution (DE) [7], the co-evolutionary algorithm (CE) [8], [9], and the particle swarm optimization (PSO) [10], [11]. These algorithms have been extensively applied in various fields such as social networks [12], signal processing [13], computer vision [14], and the intelligent manufacturing [15], [16].

Notably, evolutionary algorithms have emerged as a viable method for addressing the ES-DFJSP problem. Although existing scheduling approaches have demonstrated promising results, they primarily focus on homogeneous factory environments. Heterogeneous distributed architectures better reflect real-world workshop production environments [17], [18]. The effect of factory heterogeneity on makespan and energy consumption has not been thoroughly investigated [19]. Furthermore, the inherent randomness in evolutionary algorithms results in substantial computational overhead when applied to high-dimensional

Received 21 March 2025; revised 1 August 2025; accepted 31 August 2025. Date of publication 18 September 2025; date of current version 26 March 2026. This work was supported in part by the National Natural Science Foundations of China (NSFC) under Grant 62406104 and Grant 62472147, in part by the Key Scientific Research Project of Henan Provincial Higher Education under Grant 25A520029, and in part by the Training Program for Young Backbone Teachers in Higher Education Institutions of Henan Province of China under Grant 2025GGJS033. (Corresponding authors: Tao Li; Zidong Wang.)

Tao Li, Xingchen Li, Junna Zhang, and Dong Liu are with the College of Computer and Information Engineering, Henan Normal University, Henan 453007, China, also with the Key Laboratory of Artificial Intelligence and Personalized Learning in Education, Henan 450046, China, and also with the Engineering Lab of Intelligence Business and Internet of Things, Henan 450046, China (e-mail: litao0116@163.com; hd311tkuo215@163.com; jnzhang@htu.edu.cn; liudong@htu.edu.cn).

Zidong Wang is with the Department of Computer Science, Brunel University London, UB8 3PH Uxbridge, U.K. (e-mail: zidong.wang@brunel.ac.uk).

Recommended for acceptance by Y. Wang.

Digital Object Identifier 10.1109/TETCI.2025.3607390

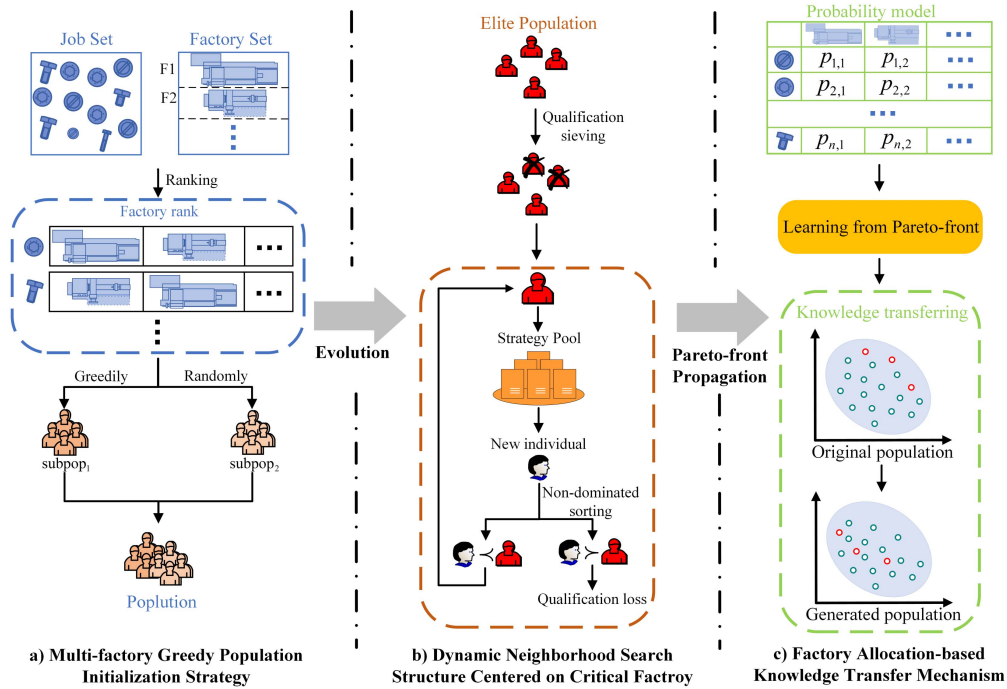


Fig. 1. Overall framework of KTCE.

or complex optimization problems. Consequently, efficiently addressing the ES-DHFJSP with both speed and accuracy remains a critical challenge in contemporary production scheduling research.

Breakthroughs in machine learning have driven growing interest in its synergy with evolutionary computation, fostering novel approaches to complex optimization problems [20], [21]. Knowledge transfer, which leverages insights from related source tasks to address target problems, has demonstrated remarkable effectiveness in optimization domains [22].

Evidence suggests that knowledge transfer mechanisms can effectively reduce computational overhead and enhance convergence rates of evolutionary algorithms. Motivated by these advantages, we propose integrating knowledge transfer techniques with evolutionary algorithms to solve the ES-DHFJSP.

Drawing on the aforementioned analysis, a knowledge transfer-based co-evolutionary search algorithm is proposed to solve the ES-DHFJSP. In the algorithm, the dynamic local search is integrated with global search to explore the optimal solution collaboratively, and the knowledge transfer is employed to minimize the makespan and the total energy consumption. The overall framework of the KTCE is shown in Fig. 1. The primary contributions of this paper are presented as follows:

- 1) A multi-factory-based greedy population initialization strategy is designed. The average processing time of jobs in each factory is adopted to generate one quarter of the initial population, which can accelerate the population convergence. Meanwhile, to ensure the diversity of the initial population, the rest of individuals are generated stochastically.
- 2) A dynamic neighborhood search structure centered on critical factory is presented. This structure dynamically

adjusts the neighborhood search around the elite population composed of elite individuals based on the degree of global search. And the search strategy is provided by a neighborhood search strategy pool which includes three neighborhood search strategies: priority factory search strategy, equal probability factory search strategy and critical path search strategy.

- 3) A factory allocation-based knowledge transfer mechanism is proposed. This mechanism utilizes the factory allocation of historical elite individuals as knowledge to construct a factory allocation probability model. New knowledge is derived from the probability model and transferred to the evolving population to accelerate the acquisition of the global optimal solution.

The rest of this paper is as follows. Section II covers related work. Section III describes the problem description of ES-DHFJSP. Section IV introduces the proposed KTCE in this paper comprehensively. In Section V, relevant experiments are designed and the experimental results are analyzed and discussed. Finally, Section VI summarizes the work of this paper and analyzes future research directions.

II. RELATED WORK

A. Distributed Flexible Scheduling Based on Evolutionary Algorithm

Evolutionary algorithms excel in parallel computing and search capabilities, rendering them highly effective for distributed flexible scheduling. Sang et al. [23] developed a high-dimensional many-objective memetic approach to address ES-DFJSP in collaborative multi-factory environments.

Li et al. [24] introduced a hybrid chemical reaction optimization algorithm for makespan minimization within scheduling schemes. Xie et al. [25] proposed a hybrid genetic tabu search algorithm, where genetic algorithms and tabu search were integrated to maintain an equilibrium in the exploration-exploitation paradigm. Yu et al. [26] constructed a knowledge-guided bi-population evolutionary algorithm, achieving enhanced convergence through knowledge-guided local search strategies and specialized evolutionary operators.

Recent research has extensively explored evolutionary approaches for distributed heterogeneous scheduling (DHS). Zhang et al. [27] introduced a biogeography-based optimization algorithm featuring local search mechanisms to address DHS with multiple process plans, employing a cosine migration rate model to escape local optima. Yang et al. [28] extracted two knowledge points from the problem structure to develop a hybrid initialization method and integrated Q-learning to dynamically adjust key parameters using population evolutionary information, resulting in accelerated the convergence. Yuan et al. [29] adopted an enhanced ϵ -greedy exploration mechanism combined with double DQN to improve training stability, thus achieving significant performance improvement.

Additionally, considerable research has focused on optimizing neighborhood search operator selection to enhance algorithmic exploration capabilities. Zhang et al. [30] introduced a novel distributed heterogeneous co-evolutionary algorithm, where learning operators and their intensities are adaptively selected based on subpopulation characteristics to improve search efficiency. Recent studies have incorporated reinforcement learning, with Q-learning and deep-Q networks being employed for optimal operator selection [31], [32], [33].

Analysis of existing literature reveals that previous research on neighborhood search strategies development continues to exhibit several limitations. These include inadequate consideration of factory prioritization in job assignment adjustments, among others. As a result, the neighborhood search exhibits limited capacity in exploring candidate optimal solutions, which adversely affects the algorithm's convergence properties. Thus, developing neighborhood search strategies with strong exploitation capabilities is essential to enhance the algorithm's search performance.

Furthermore, computational resource allocation serves as another critical factor affecting algorithm performance. However, existing studies predominantly focus on selecting optimal neighborhood search strategies while largely neglecting in-depth investigation into resource allocation mechanisms. Consequently, we develop a dynamic framework that adjusts the intensity of neighborhood search and restricts its application exclusively to promising individuals.

B. Knowledge Transfer

In recent years, knowledge transfer has emerged as an effective optimization mechanism with numerous researchers exploring its integration with evolutionary algorithms. Zhang et al. [34] incorporated knowledge transfer into genetic programming, implementing task-oriented knowledge sharing within crossover

operations to enhance algorithmic convergence. Lin et al. [35] developed knowledge transfer prediction and maintenance sampling techniques to mitigate negative transfer effects while improving optimization efficiency. Zheng et al. [36] integrated CE with fuzzy deep transfer learning for predicting relief demand.

Furthermore, historical experience, neighborhood information, and elite individuals are frequently utilized as knowledge sources for transfer to populations, thus enhancing optimization efficiency. Jiang et al. [37] formulated a dynamic multi-objective evolutionary algorithm that utilizes historical experience to construct effective initial population pools. Wang et al. [38] introduced an EA based on decomposition with dual neighborhoods, enhancing the transfer effectiveness through acquiring knowledge from the task's neighborhood. Guo et al. [39] developed a knowledge pool approach that preserves Pareto-optimal solutions from historical environments. When encountering new environments, this approach selectively applies the most relevant knowledge to initialize populations, facilitating rapid convergence to Pareto-optimal solutions. Zhang et al. [40] implemented bidirectional knowledge transfer between dual populations (POP) and elite archives (ERT), where population diversity is maintained through inter-population individual migration while evolutionary progress is enhanced by exchanging elite solutions between populations. Peng et al. [41] proposed a micro many-objective evolutionary algorithm with knowledge transfer (μ MaOEA) that enhances the performance of unoptimized niches by transferring neighborhood niches knowledge, thereby fostering the emergence of better individuals.

Mitigating negative transfer remains a critical challenge in knowledge transfer optimization. Xu et al. [42] developed an adaptive EMTO framework that dynamically adjusts knowledge transfer parameters, including transfer frequency, source selection, and transfer intensity. This approach effectively maintains the equilibrium between intratask evolutionary processes and intertask knowledge transfer. Wang et al. [43] proposed a task-specific anomaly detection framework to evaluate inter-task relationships, enabling the identification and filtering of potentially detrimental knowledge transfers.

It can be seen that the combination of knowledge transfer and evolutionary algorithms can utilize the historical experience to quickly generate appropriate high-quality population when the new environment emerges. In this way, the convergence speed of the algorithm in new environments can be improved. For this reason, we consider combining knowledge transfer and evolutionary algorithms to solve ES-DHFJSP. The historical optimization experience is leveraged to guide population evolution, significantly enhancing the algorithm's efficiency.

III. PROBLEM DESCRIPTION

ES-DHFJSP incorporates multiple complex constraints that significantly increase its computational complexity. The problem is characterized by heterogeneous factories, each possessing varying numbers of machines, while individual operations may require different sets of available machines with distinct processing times. Formally, the ES-DHFJSP involves processing a set of N jobs across n_f heterogeneous factories, where each

TABLE I
SYMBOLS AND DESCRIPTIONS

Symbol	Description
N	total number of jobs.
I	the set of jobs.
n_f	total number of heterogeneous factories.
F	the ensemble of heterogeneous factories and $F = \{1, \dots, n_f\}$.
m_f	the number of machines of factory F_f .
M_f	the set of machines of factory F_f and $M_f = \{1, \dots, m_f\}$.
M^n	the vector of the number of machines for each factory.
n_i	the number of operations of the job I_i .
J_i	the set of operations of job I_i and $J_i = \{1, \dots, n_i\}$.
J^n	the vector of the number of operations for each job.
$O_{i,j}$	the j -th operation of I_i .
$M_{i,j,f}$	the set of available machines of $O_{i,j}$ in F_f .
$M_{i,j,f}^n$	the number of available machines of $O_{i,j}$ in F_f .
$T_{i,j,f,m}$	the processing time of the operation $O_{i,j}$ on the m -th machine in F_f .
$x_{i,j,f,m}$	$x_{i,j,f,m}$ is a bool decision variable. If $O_{i,j}$ is processed on the m -th machine in F_f , $x_{i,j,f,m} = 1$; otherwise, $x_{i,j,f,m} = 0$.
$\varphi_{i,j,f,m}$	$\varphi_{i,j,f,m}$ is a bool decision variable. If the m -th machine in F_f is included in $M_{i,j,f}$, $\varphi_{i,j,f,m} = 1$; otherwise, $\varphi_{i,j,f,m} = 0$.
$u_{i,j,f,m,p}$	$u_{i,j,f,m,p}$ is a bool decision variable. If $O_{i,j}$ is processed at the p -th position on the m -th machine in F_f , $u_{i,j,f,m,p} = 1$; otherwise, $u_{i,j,f,m,p} = 0$.
$v_{i,f}$	$v_{i,f}$ is a bool decision variable. If I_i is assigned to F_f , $v_{i,f} = 1$; otherwise, $v_{i,f} = 0$.
W_I	the idle power of all machines.
W_P	the process power of all machines.
$P_{f,m}$	the number of positions of the m -th machine in F_f . Each position corresponds to an operation.
$ST_{i,j,f}$	the start processing time of $O_{i,j}$ in F_f .
$FT_{i,j,f}$	the finish processing time of $O_{i,j}$ in F_f .
$SP_{f,m,p}$	the start processing time of the p -th position on the m -th machine in F_f .
$FP_{f,m,p}$	the finish processing time of the p -th position on the m -th machine in F_f .

job consists of multiple operations and each heterogeneous factory contains m_f machines. The notation and terminology used throughout this paper are defined in Table I. The following assumptions are established for problem formulation:

- 1) Initially, all factories and machines are accessible, and each job can be assigned in any factory.
- 2) Job assignments to factories are final and cannot be altered.
- 3) A machine cannot process two operations simultaneously and be interrupted during the machining process.
- 4) Operations cannot be split across multiple machines.
- 5) For each job, the next operation can only be processed only when the previous operation is finished.
- 6) All machines in all factories are started from the beginning. When the assigned operations are completed, the machine will immediately stop running.

ES-DHFJSP aims to find an optimal scheduling solution [44]. In this paper, the two objectives of makespan and total energy consumption (TEC) are mainly considered.

- 1) F_1 : Makespan refers to the time required to complete the processing of all jobs, which can directly reflect the efficiency of industrial production. Makespan is

calculated as:

$$\min F_1 = C_{\max} = \max FT_{i,j,f} \quad (1)$$

where $i \in I$, $j \in J_i$, and $f \in F$.

- 2) F_2 : TEC refers to the total energy consumption required to complete the processing of all jobs, which can directly reflect the cost of industrial production. TEC is calculated as:

$$\min F_2 = TEC = P_1 + P_2 \quad (2)$$

$$P_1 = \sum_{i=1}^N \sum_{j=1}^{n_i} \sum_{f=1}^{n_f} \sum_{m=1}^{m_f} T_{i,j,f,m} \cdot W_P \cdot x_{i,j,f,m} \cdot \varphi_{i,j,f,m} \quad (3)$$

$$P_2 = \sum_{f=1}^{n_f} \sum_{m=1}^{m_f} \sum_{p=1}^{P_{f,m}} (SP_{f,m,p} - FP_{f,m,p-1}) \cdot W_I \quad (4)$$

In addition, ES-DHFJSP has the following constraints:

$$\sum_{f=1}^{n_f} \sum_{m=1}^{m_f} x_{i,j,f,m} * \varphi_{i,j,f,m} = 1, \forall i \in I, j \in J_i \quad (5)$$

$$\sum_{f=1}^{n_f} v_{i,f} = 1, \forall i \in I \quad (6)$$

$$\sum_{i=1}^N \sum_{j=1}^{n_i} u_{i,j,f,m,p} \leq 1, \forall f \in F, m \in M_{i,j,f}, p = \{1, 2, \dots, P_{f,m}\} \quad (7)$$

$$\sum_{i=1}^N \sum_{j=1}^{n_i} u_{i,j,f,m,p+1} \leq \sum_{i=1}^N \sum_{j=1}^{n_i} u_{i,j,f,m,p}, \forall f \in F, m \in M_{i,j,f}, p = \{1, 2, \dots, P_{f,m} - 1\} \quad (8)$$

$$FT_{i,j,f} \leq ST_{i,j+1,f}, \forall i \in I, j = \{1, 2, \dots, n_i - 1\}, f \in F \quad (9)$$

$$FP_{f,m,p} \leq SP_{f,m,p+1}, \forall f \in F, m \in M_f, p = \{1, 2, \dots, P_{f,m} - 1\} \quad (10)$$

where (5) stipulates that any operation can only be processed by one available machine. Equation (6) ensures that any job can only be processed in the same factory. Equation (7) indicates that any machine can only process one operation at a time. Equation (8) describes that each position can be used only after the previous position has been used. Equation (9) states that the current operation can only start after the previous operation is completed. Equation (10) ensures that all operations on a machine must be completed in the specified order.

IV. OUR APPROACH: KTCE

This section presents an in-depth explanation of the proposed KTCE algorithm, encompassing its core methodology, the encoding and decoding mechanisms, the greedy population

Algorithm 1: KTCE algorithm.

Input: Population size ps , crossover rate Pc , mutation probability Pm , maximum number of function evaluations $MaxNFES$, maximum number of neighborhood searches ε , learning intensity σ .

Output: Pareto-optimal solutions set.

- 1 Initialize current evaluation count $NFES = 0$, elite archive $\mathcal{A} = \emptyset$, qualification vector $Qual = \emptyset$, current iteration count $t = 0$, factory allocation probability model PM , and Pareto front $PF_0 = \emptyset$.
- 2 $P_0 \leftarrow$ Generate initial population by greedy initialization strategy.
- 3 **while** $NFES \leq MaxNFES$ **do**
- 4 $t = t + 1$.
- 5 **if** meet the transfer condition **then**
- 6 $P_0 \leftarrow$ Factory allocation-based knowledge transferring(PM, P_0).
- 7 **end**
- 8 $O \leftarrow$ Crossover and mutation(P_0, Pc, Pm).
- 9 $P', PF \leftarrow$ Obtain the new population P' and the Pareto front PF by Environmental Selection($P_0 \cup O$).
- 10 $\mathcal{A} \leftarrow$ Get the Pareto front($\mathcal{A} \cup PF$).
- 11 $Qual \leftarrow$ Update qualification vector.
- 12 $PM \leftarrow$ Factory Allocation-based knowledge learning($PF/PF_0, PM, \sigma$).
- 13 $PF', P'' \leftarrow$ Dynamic neighborhood search based on critical factory($\mathcal{A}, \varepsilon, Qual, P', NFES, MaxNFES, \sigma$).
- 14 $\mathcal{A} \leftarrow PF', P_0 \leftarrow P'', PF_0 \leftarrow PF'$.
- 15 **end**

initialization strategy, dynamic neighborhood search structure based on critical factory, and knowledge transfer mechanism utilizing factory allocation probability models. Additionally, a comprehensive analysis of the algorithm's computational complexity is provided.

A. The Main Idea of KTCE

The KTCE algorithm operates through the following key phases. Initially, a multi-factory-based greedy population initialization strategy is employed to generate superior initial individuals. Subsequently, a factory allocation-based knowledge transfer mechanism guides population optimization. The algorithm then executes global search across the entire population, while implementing neighborhood search within an elite population $\mathcal{A}$ comprising superior individuals. Finally, a dynamic neighborhood search structure centered on critical factory is utilized, which evaluates the current Pareto front using qualification vector $Qual$ and conducts adaptive neighborhood exploration. This framework enables thorough investigation of promising solution regions. The collaboration between dual populations enhances the algorithm's capability to identify optimal solutions. The complete algorithmic procedure is detailed in Algorithm 1.

B. Encoding and Decoding Schema

1) *Encoding Schema:* This paper employs a three-layer vector $s = [OS, MS, FA]$ as the encoding schema. OS determines the processing order, and its length is equivalent to operations count SH , where $SH = \sum_{i=1}^N n_i$. Job numbers are assigned to

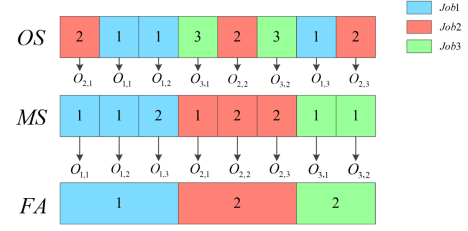


Fig. 2. Example for encoding schema in the KTCE.

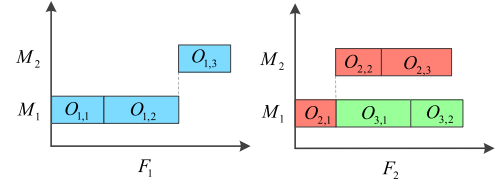


Fig. 3. Example for decoding schema in the KTCE.

elements, with each number pointing to a specific job operation. MS represents the machine selection for each operation, and its length is equal to OS. The k -th element indicates the machine number for processing the corresponding operation. FA prescribes the processing factory for each job, and its length is N . The i -th element denotes the factory number assigned to the i -th job. A encoding situation is shown in Fig. 2, where $N = 3$, $J^n = \{3, 3, 2\}$, $n_f = 2$, $M^n = \{2, 2\}$, $SH = \sum_{i=1}^3 n_i = 8$.

2) *Decoding Schema:* Firstly, the jobs are assigned to the corresponding factory according to the FA. And then the processing machine is selected for each job by the MS vector. Finally, the operation sequence on each machine is determined based on OS to form a complete scheduling scheme. Fig. 3 shows the scheduling scheme obtained after decoding.

C. Multi-Factory-Based Greedy Population Initialization Strategy

In evolutionary computation, the quality and uniformity of the initial population significantly influence the efficiency of solution space exploration and algorithmic convergence rate [45]. Hence, the development of effective population initialization strategies is crucial for solving optimization problems. This paper proposes a multi-factory-based greedy population initialization strategy that constructs superior initial population while preserving the diversity.

First, the average total processing time $AT_{i,f}$ of any job across all factories is computed by:

$$AT_{i,f} = \sum_{j=1}^{n_i} \frac{\sum_{m \in M_{i,j,f}} T_{i,j,f,m}}{M_{i,j,f}^n} \quad (11)$$

The factories are arranged in ascending order according to $AT_{i,f}$. Subsequently, i -th job is assigned to the factory with the minimum $AT_{i,f}$. This process repeats until all jobs are assigned. This greedy initialization approach facilitates the generation of high-quality FA chromosomes while enhancing algorithm convergence. For example, given the current factory set $F =$

$\{1, 2, 3, 4, 5\}$, average total processing times for job I_i at each factory are calculated as $AT = \{19.6, 17.0, 31.2, 24.2, 21.4\}$. Therefore, factory F_2 achieves the minimum average total processing time. Under the greedy strategy, I_i is allocated to F_2 . Alternatively, the randomized strategy assigns I_i to any factory in F with equal probability.

However, excessive use of greedy strategies may lead to limited search range of the initial population, resulting in falling into local optima. FA vectors of one quarter of the initial population are generated by the proposed greedy strategy, while the remaining are generated by the random strategy.

Following population initialization, a global search with extended step size is implemented to expand the exploration space and identify promising solution regions. The algorithm employs two crossover operators [46]: Priority Operation Crossover (POX) for operation sequence crossover, and Universal Crossover (UX) for machine selection and factory allocation crossover. Subsequently, two mutation strategies are applied according to probability Pm : 1) position exchange between two randomly selected operations in the operation sequence; and 2) reassignment of processing machines for randomly selected operations to alternative available machines. Environmental selection is performed using non-dominated sorting [47].

D. Dynamic Neighborhood Search Structure Centered on Critical Factory

While global search with large step sizes effectively expands the solution space, identifying optimal solutions within extensive search domains under time constraints remains challenging, particularly when exploring regions around specific solution targets. Neighborhood search methods have emerged as an effective approach and have been extensively investigated [48], [49], [50]. Consequently, neighborhood search techniques are essential for conducting intensive exploration around elite solutions. The synergistic integration of global and neighborhood search mechanisms substantially enhances the algorithm's capability to identify optimal solutions.

In the context of ES-DHFJSP optimization, critical factory and critical path factors play vital roles. Their definitions are as follows:

- The critical factory is defined as the factory with the maximum completion time.
- The critical path refers to the sequence of operations requiring the longest total processing time from start to end.

Accordingly, three neighborhood search strategies combined with a dynamic search structure are developed to maintain an equilibrium between neighborhood and global search operations. These local search strategies are detailed as follows:

1) LS_1 (Neighborhood Search Strategy Based on Priority Factory): The neighborhood strategy focuses on the adjustment of factory assignment for the critical factory. Firstly, calculate the finish time of factories other than the critical one and rank them in ascending sequence, where the higher the ranking of the factory, the smaller the makespan, and the higher the priority. Then select a job at random from the critical factory and add it to the factory with the highest priority.

2) LS_2 (Neighborhood Search Strategy Based on Equal Probability Factory): This neighborhood strategy also focuses on the adjustment of the critical factory. It concentrates on slightly increasing the search space to explore solutions more effectively in the neighborhood. LS_2 is to select a factory with moderate probability from other factories except for the critical factory. Then, a job is randomly extracted from the critical factory and added to the chosen factory.

3) LS_3 (Neighborhood Search Strategy Based on Critical Path): This neighborhood strategy focuses on the adjustment of the critical path, which exchanges the positions of two operations randomly.

It should be emphasized that excessive allocation of computational resources to neighborhood search may result in premature convergence to local optima. Therefore, efficient resource allocation between neighborhood and global search operations is crucial for enhancing algorithmic performance [51], [52]. This paper introduces a dynamic neighborhood search framework that adaptively distributes computational resources based on search requirements at different optimization stages.

First, two variables are defined as: (1) LST represents the frequency of neighborhood searches; (2) $maxLST$ denotes the current maximum allowable number of neighborhood searches, calculated as follows:

$$maxLST = \left\lceil \varepsilon \times \frac{NFES}{MaxNFES} \right\rceil \quad (12)$$

During the initial phase of KTCE, computational resources are predominantly allocated to global search for broad exploration. As the algorithm progresses through iterations, the effectiveness of large-scale exploration diminishes, necessitating the allocation of computational resources to neighborhood search around elite solutions for identifying promising optimal candidates.

Additionally, a qualification vector $Qual$ is introduced to filter individuals eligible for neighborhood search. It has the same size as $\mathcal{A}$, and both are initially empty. Upon adding a new solution p_i to $\mathcal{A}$, $Qual_{p_i}$ is added to $Qual$ and initialized to 1. Correspondingly, when p_i is removed from $\mathcal{A}$, $Qual_{p_i}$ is removed. For solution p_i in $\mathcal{A}$, neighborhood search is activated only when $Qual_{p_i} = 1$.

Specifically, p'_i is defined as the obtained individual through neighborhood search. end_f denotes the termination indicator and end_{max} represents the maximum threshold value of end_f , calculated as follows:

$$end_{max} = \left\lceil maxLST \times \frac{2}{3} \right\rceil \quad (13)$$

When $end_f > end_{max}$, indicating that end_f has exceeded its upper bound, the neighborhood search performed on solution p_i fails to yield a better solution. Then, $Qual_{p_i} = Qual_{p_i} - 1$. Simultaneously, p_i is excluded from subsequent neighborhood search iterations. This mechanism significantly reduces redundant exploitation.

Algorithm 2 presents the pseudo-code of the dynamic neighborhood search structure based on critical factory.

Algorithm 2: Dynamic Neighborhood Search Structure Centered on Critical Factory.

Input: Elite population $\mathcal{A}$, maximum number of neighborhood searches ε , Qualification vector $Qual$, population P' , current evaluation count $NFEs$, maximum number of function evaluations $MaxNFEs$.

Output: $\mathcal{A}'$, P' .

```

1  $L \leftarrow Len(\mathcal{A})$ .
2 for  $i=1$  to  $L$  do
3    $p_i \leftarrow \mathcal{A}(i)$ ,  $end_f = 0$ ,  $maxLST = \lceil \varepsilon \times \frac{NFEs}{MaxNFEs} \rceil$ .
4   if  $Qual_i = 0$  then
5     Continue.
6   end
7   while  $LST \leq maxLST$  do
8     if  $end_f > end_{max}$  then
9        $Qual_i \leftarrow Qual_i - 1$ , break.
10    end
11     $p'_i \leftarrow$  Randomly select a strategy from the strategy pool for neighborhood search( $p_i$ ).
12    if  $p'_i$  dominates  $p_i$  then
13       $\mathcal{A} \leftarrow \mathcal{A} \cup p'_i$ .
14      if  $rand() < 0.5$  then
15         $P' \leftarrow$  Select an individual from the final front of  $P'$  randomly to replace with  $p'_i$ .
16      else
17         $P' \leftarrow$  Select an individual from  $P'$  randomly to replace with  $p'_i$ .
18      end
19       $end_f = 0$ .
20    else if  $p_i$  and  $p_i$  do not dominate each other then
21       $p'_i \leftarrow$  Randomly select from  $p'_i$  and  $p_i$ .
22       $\mathcal{A} \leftarrow \mathcal{A} \cup p'_i$ .
23       $P' \leftarrow$  Replace an individual in the population as  $p'_i$  dominates  $p_i$ .
24       $end_f = 0$ .
25    else
26       $end_f = end_f + 1$ .
27    end
28  end
29 end
30  $\mathcal{A}' \leftarrow$  non-dominated sort( $\mathcal{A}$ ).
31  $PM \leftarrow$  Factory Allocation-based Knowledge Learning( $\mathcal{A}'/\mathcal{A}, PM, \sigma$ ).

```

E. Factory Allocation-Based Knowledge Transfer Mechanism

Beyond determining operation sequences and machine assignments, ES-DHFJSP necessitates factory allocation decisions, substantially expanding the problem dimensionality. Consequently, we partition the solution space into multiple subspaces based on factory allocation patterns, with each subspace representing a distinct allocation strategy. Upon identifying elite solutions, selected individuals are directed to explore promising regions within these elite solution subspaces.

To facilitate this approach, we develop a knowledge transfer mechanism utilizing a factory allocation probability model. This framework leverages historical elite solutions' factory assignments as transferable knowledge to guide the current population toward promising subspaces. This mechanism enhances both

Algorithm 3: Factory Allocation-based Knowledge Learning.

Input: the newly added elite individuals set $Diff_{t+1}$, factory allocation probability model PM , learning intensity σ .

Output: PM .

```

1  $L \leftarrow Len(Diff_{t+1})$ .
2 for  $i=1$  to  $L$  do
3   while  $j \leq N$  do
4      $PM \leftarrow$  Update the probability model by equation (15).
5   end
6 end

```

population convergence rate and solution quality. To ensure effective subspace exploration, knowledge transfer is implemented at regular T -generation intervals, where $ps/10$ individuals are selected for factory allocation adjustment using the probability model, thereby concentrating the search within elite solution subspaces. The detailed implementation of this factory allocation-based knowledge transfer mechanism proceeds as follows:

1) *Factory Allocation-based Knowledge Learning:* Let $PM \in \mathbb{R}^{N \times (n_f + 1)}$ denote the factory allocation probability model. Initially, all factories share a uniform probability distribution. Specifically, for any job I_i , $PM_i = [0, \frac{1}{n_f}, \frac{2}{n_f}, \dots, 1]$. Assume the elite population is denoted as PF_t after the completion of the t -th iteration. Then, a new Pareto front PF_{t+1}^1 is generated after the $(t+1)$ -th global search, and the newly added elite individuals are collected to form the set $Diff_{t+1}$, it is shown as:

$$Diff_{t+1} = PF_{t+1}^1 / PF_t \quad (14)$$

Then, the FA encoding of each elite individual $d_i \in Diff_{t+1}$ is obtained, and it is denoted as dFA_i . Afterwards update the factory allocation probability model PM based on dFA_i . The update formula is shown as:

$$PM_{j,f} = \begin{cases} PM_{j,f} + \sigma \times (n_f - f - 1) & \text{if } f \geq dFA_{i,j} \\ PM_{j,f} - \sigma \times (f + 1), & \text{otherwise} \end{cases} \quad (15)$$

where σ represents the learning intensity, j is the job index, and $dFA_{i,j}$ denotes the FA encoding of the j -th job in individual dFA_i . In Algorithm 3, the pseudo-code for factory allocation-based knowledge learning is outlined.

2) *Factory Allocation-based Knowledge Transferring:* First, the vector $Q = [P_1, P_2, \dots, P_r]$ is obtained based on the non-dominated sorting of the population, where P_i represents the vector of individuals with Pareto rank i , and the highest rank is denoted as r . Then, select individuals of this rank as transfer individuals starting from P_r . If the number of individuals is insufficient, the individuals from the previous rank are selected until the number of extracted individuals reaches $ps/10$. The factory assignments of each transfer individual are adjusted according to the PM .

The pseudo-code of factory allocation-based knowledge transferring is shown in Algorithm 4.

Algorithm 4: Factory Allocation-based Knowledge Transferring.

Input: factory allocation probability model PM , current population P .
Output: P .

```

1  $Q, r \leftarrow \text{Fast Nondominated Sort}(P)$ .
2  $TS \leftarrow \text{Select transfer individuals}$ .
3  $L \leftarrow \text{Len}(TS)$ ,  $P' \leftarrow P/TS$ .
4 for  $l=1$  to  $L$  do
5   for  $i=1$  to  $N$  do
6     Generate a random number  $rd$  range in  $[0,1)$ .
7     Reassign factories of job  $I_i$  based on which interval
       of  $PM$  the value  $rd$  falls into.
8   end
9   Update  $TS_i$ .
10 end
11  $P \leftarrow P' \cup TS$ .
```

F. Time Complexity of KTCE

The KTCE algorithm comprises the following three components: evolutionary process, neighborhood search and knowledge transfer. The complexity of each component is described as follows:

- 1) *Evolutionary Process*: Within the global search, the complexity are decomposed into two distinct phases: $O(ps)$ for the crossover-mutation phase and $O(ps^2)$ for the environmental selection phase. Therefore, the evolutionary process requires $O(ps^2)$.
- 2) *Neighborhood Search*: The neighborhood search section has a time complexity of $O(\text{size}(\mathcal{A}) * (\max LST + N))$. The learning section requires $O(\text{size}(\mathcal{A}) * N)$. Hence, the total complexity of the neighborhood search section is $O(\text{size}(\mathcal{A}) * (\max LST + N))$.
- 3) *Knowledge Transfer*: The time complexity of knowledge transfer stage is $O((ps + r) * ps)$.

We define *Iter* as iteration count. Hence, the aggregate complexity of the KTCE is $O(ps^2 * \text{Iter})$.

V. EXPERIMENT RESULTS AND ANALYSIS

The current section conducts a in-depth experimental evaluation of the KTCE algorithm. The experimental analysis is structured into five subsections: first, CPLEX is employed to verify the ES-DHFJSP model; second, the benchmark instances and performance metrics are introduced; third, parameter calibration procedures are detailed; furthermore, to execute individual component, ablation experiments are carried out; finally, comparative analyses against state-of-the-art multi-objective optimal algorithms are performed. KTCE is programmed using Python 3.9. All algorithms run on an Intel(R) Core (TM) i5-10210 U CPU @ 1.60 GHz 2.11 GHz and 8 GB of RAM.

A. Model Verification

To verify the correctness of the ES-DHFJSP model, CPLEX 22.1.0 (full version) is employed. The model is coded in the native optimization language of CPLEX and verified on six generated small-scale instances. Each instance contains 4-6 jobs,

TABLE II
COMPARISON RESULTS BETWEEN KTCE AND CPLEX ON SIX SMALL-SCALE INSTANCES

Instance	makespan		TEC		time(s)	
	CPLEX	KTCE	CPLEX	KTCE	CPLEX	KTCE
4J2F	41	41	625	625	0.590	1.428
4J3F	43	43	648	648	0.838	0.992
5J2F	45	45	806	806	3.232	1.609
5J3F	44	44	766	766	0.281	1.638
6J2F	55	55	1032	1032	19.120	2.938
6J3F	48	48	886	886	29.196	3.290

each having 5 operations, along with 2 or 3 factories, each equipped with 5 machines. The maximum runtimes of CPLEX and KTCE is limited to 600 seconds. Utilizing the optimal solution obtained by CPLEX as the benchmark, the time for KTCE to reach the benchmark is recorded. To minimize variability, we executed 10 independent runs on each instance for both CPLEX and KTCE. The average computation times are computed as the final times. The computational results are presented in Table II. The results verify the correction of the model. Furthermore, they demonstrate that KTCE converges to the optimal solution within a desirable time.

B. Instances and Metrics

The evaluation of KTCE is conducted on 20 instances of varying scales. The job count N takes values from $\{10, 20, 30, 40, 50, 100, 150, 200\}$, while the factory count n_f spans the interval $[2, 7]$. All factories contain $m = 5$ machines. In addition, any machine consumes 4kWh for processing and 1kWh for idle states. The instances are available at <https://wewnyin.github.io/wenyingong/Software/DQCE-code.zip>.

Two standard performance metrics are employed to evaluate the multi-objective evolutionary algorithms (MOEAs): hypervolume (HV) and inverted generational distance (IGD). The HV metric quantifies the algorithm's comprehensive performance by computing the hypervolume bounded by the Pareto front and a reference point, where elevated HV indicates superior overall performance. The IGD metric characterizes comprehensive performance through the distance separating the true and approximated Pareto fronts, with smaller IGD values suggesting better convergence and solution diversity.

C. Parameter Sensitivity Analysis

The performance of KTCE is greatly affected by the parameter settings, making it essential to investigate the best combination. Therefore, the Taguchi experimental design method [53] is applied to identify the optimal parameters of the KTCE. The KTCE encompasses several critical parameters, including population size ps , crossover probability Pc , mutation probability Pm , learning intensity σ , and upper bound for neighborhood search ε . In here, we set each parameter at four levels: $ps = \{60, 80, 100, 120\}$, $Pc = \{0.7, 0.8, 0.9, 1.0\}$, $Pm = \{0.05, 0.1, 0.15, 0.2\}$, $\sigma = \{0.01, 0.02, 0.03, 0.04\}$, and $\varepsilon = \{3, 4, 5, 6\}$.

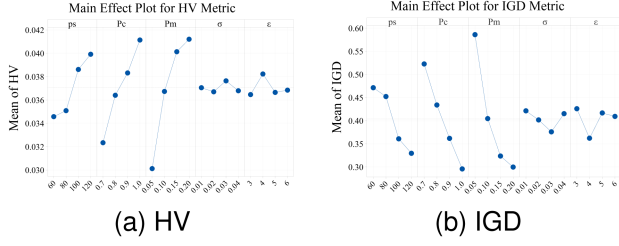


Fig. 4. Influence trend of parameters in the KTCE on HV and IGD.

TABLE III
COMPARISONS OF KTCE AND THE VARIANTS ON HV METRICS

Ins	KTCE1		KTCE2		KTCE3		KTCE
	mean	p-value	mean	p-value	mean	p-value	
10J2F	0.0764	4.5755E-01	0.0649	1.3756E-01	0.0785	3.5807E-01	0.0755
20J2F	0.0414	4.6853E-01	0.0410	4.8047E-01	0.0397	3.2142E-01	0.0411
20J3F	0.0558	3.7596E-01	0.0564	3.0011E-01	0.0620	8.4189E-02	0.0540
30J2F	0.0371	1.1775E-01	0.0385	2.3822E-01	0.0310	3.5849E-04	0.0398
30J3F	0.0550	1.7452E-01	0.0522	2.9852E-01	0.0454	2.4835E-02	0.0511
40J2F	0.0334	3.3616E-01	0.0371	1.0383E-01	0.0293	4.8375E-02	0.0344
40J3F	0.0431	4.3212E-01	0.0479	6.0814E-02	0.0336	1.1151E-03	0.0435
40J4F	0.0510	4.8273E-01	0.0456	3.0149E-02	0.0326	1.2882E-04	0.0509
50J3F	0.0569	2.0164E-01	0.0546	4.8041E-01	0.0383	2.5840E-04	0.0544
50J4F	0.0459	8.8428E-02	0.0519	2.7624E-01	0.0346	2.3158E-04	0.0505
50J5F	0.0614	6.2963E-02	0.0487	6.8616E-02	0.0421	7.1813E-03	0.0550
100J4F	0.0465	2.9029E-05	0.0591	3.3706E-01	0.0210	6.4334E-09	0.0584
100J5F	0.0425	4.6175E-05	0.0433	8.0428E-03	0.0169	3.1376E-11	0.0524
100J6F	0.0572	4.1538E-04	0.0706	3.4618E-01	0.0199	8.3332E-14	0.0718
100J7F	0.0548	2.2510E-08	0.0771	4.1754E-01	0.0280	3.3860E-08	0.0765
150J5F	0.0504	6.8750E-07	0.0609	4.6830E-02	0.0159	5.7506E-16	0.0648
150J6F	0.0567	5.4032E-05	0.0659	3.3892E-02	0.0199	2.7602E-10	0.0694
150J7F	0.0595	3.0644E-05	0.0727	4.9159E-01	0.0225	8.1750E-08	0.0727
200J6F	0.0581	2.7475E-07	0.0714	8.4895E-02	0.0195	3.8152E-08	0.0738
200J7F	0.0667	2.0321E-05	0.0687	3.5910E-04	0.0229	9.2921E-09	0.0778
-/+	9/11/0		5/15/0		17/3/0		

An orthogonal table $L_{16}(4^5)$ is utilized for the parameter experiment, and each experiment is adhered to the same termination condition: the maximum evaluation count $MaxNFES = 200 \times \sum_{i=1}^N n_i$. Each parameter setting runs 10 times to derive the average of two metrics. Fig. 4 illustrates the evaluation metrics across the five parameters at each level. Consequently, the selected parameter combination for the KTCE is $ps = 120$, $Pc = 1.0$, $Pm = 0.2$, $\sigma = 0.03$, $\epsilon = 4$.

D. Ablation Experiment

To assess individual algorithmic component's contribution, three variant algorithms are constructed: 1) KTCE1, which excludes the greedy population initialization strategy; 2) KTCE2, which omits the dynamic neighborhood search structure; and 3) KTCE3, which eliminates the knowledge transfer mechanism. To ensure fair comparative analysis, all variants and the complete KTCE algorithm are executed independently 10 trials across 20 test instances, with the termination condition set as $MaxNFES = 200 \times \sum_{i=1}^N n_i$.

Tables III and IV respectively show comparative results of KTCE versus three variant algorithms across two performance metrics, accompanied by p-values derived from t-tests ($\alpha = 0.05$). “-” indicates that the comparative algorithm shows a clear disadvantage compared to KTCE, “=” signifies comparable performance, and “+” denotes that the comparative variant is markedly superior to the KTCE. From the experiments, the

TABLE IV
COMPARISONS OF KTCE AND THE VARIANTS ON IGD METRICS

Ins	KTCE1		KTCE2		KTCE3		KTCE
	mean	p-value	mean	p-value	mean	p-value	
10J2F	0.2299	2.5099E-01	0.2776	4.8533E-01	0.1331	1.1897E-02	0.2751
20J2F	0.3475	4.6322E-01	0.3521	4.3706E-01	0.3900	2.2547E-01	0.3404
20J3F	0.3283	3.2425E-01	0.3387	2.0926E-01	0.2384	2.2603E-01	0.2920
30J2F	0.3677	2.9047E-03	0.2069	4.6889E-01	0.5836	1.7968E-04	0.2101
30J3F	0.3000	2.9560E-01	0.4447	1.7730E-02	0.4498	4.8789E-02	0.3299
40J2F	0.3264	2.6641E-02	0.2185	4.9868E-01	0.4959	7.4489E-03	0.2184
40J3F	0.2435	5.9029E-02	0.2199	2.0638E-01	0.4761	6.7411E-05	0.1877
40J4F	0.2367	1.4549E-01	0.2634	3.9529E-02	0.5939	1.1934E-04	0.2125
50J3F	0.2292	2.0287E-01	0.2921	2.7400E-01	0.6319	2.6576E-04	0.2682
50J4F	0.3621	2.3762E-03	0.2173	1.5421E-01	0.5487	1.1851E-04	0.1796
50J5F	0.2550	1.7297E-01	0.3405	4.1045E-03	0.5634	2.3007E-04	0.2252
100J4F	0.3198	8.1988E-03	0.1442	1.3741E-03	0.8789	3.1232E-07	0.2401
100J5F	0.3581	1.1968E-06	0.2697	7.0566E-02	0.9159	4.0141E-08	0.1943
100J6F	0.3502	7.8589E-05	0.1897	9.2063E-03	1.0019	1.8546E-09	0.1199
100J7F	0.4469	9.4117E-09	0.2185	1.1212E-02	0.9245	2.7121E-07	0.1405
150J5F	0.3372	6.0635E-06	0.1723	1.7869E-01	1.1242	1.3114E-14	0.1409
150J6F	0.3431	5.1757E-06	0.1870	1.5973E-02	1.0467	7.2997E-08	0.1211
150J7F	0.3453	1.9141E-08	0.1778	1.1220E-01	0.9229	1.4665E-06	0.2128
200J6F	0.2995	1.3362E-07	0.1268	9.8222E-02	1.0372	4.2289E-07	0.1022
200J7F	0.2552	2.7880E-06	0.2094	1.3180E-03	0.9577	2.8110E-07	0.1257
-/+	12/8/0		7/12/1		17/2/1		

TABLE V
PARAMETERS SETTING FOR COMPARISON ALGORITHMS

Algorithms	Parameters
MOEA/D	$T = 10$
LRVMA	$\gamma = 0.8, \alpha = 0.1, \epsilon = 0.95$
DQCE	$\alpha = 0.001, bs = 16, \epsilon = 0.9, \gamma = 0.9, S_E = 512$
MFLEDA	$\beta_{max} = 0.5, \ell = 10$

knowledge transfer mechanism significantly enhances algorithmic efficiency. And the other two are slightly inferior to the knowledge transfer mechanism, but they have also achieved satisfactory results.

E. Comparison and Analysis

To highlight KTCE's performance advantages, it is compared with MOEA/D [54], NSGA-II [47], TSKEA [55], LRVMA [56], DQCE [57] and MFLEDA [58]. These algorithms leverage varied approaches to tackle ES-DHFJSP effectively. MOEA/D applies decomposition in its framework. NSGA-II relies on non-dominated sorting. TSKEA, a two-stage method, incorporates knowledge-driven. LRVMA and DQCE leverage reinforcement learning and deep reinforcement learning, respectively. MFLEDA utilizes a probabilistic modeling approach to produce new solutions. DQCE is implemented in Python 3.9, while other algorithms are implemented in MATLAB 2022a.

The parameter setting for the KTCE follows the optimal values obtained in subsection V-C. For the comparative algorithms, the population size ps , the probabilities for crossover (Pc) and mutation (Pm) are set the same as the original. And, other parameters are shown in Table V, where T is the size of neighborhood in MOEA/D. In LRVMA, γ is discount factor, while include α (learning rate) and ϵ (greedy factor). α and ϵ in DQCE have the same meaning as in LRVMA, and there are batch size bs and experience pool size S_E . In addition, β_{max} and ℓ represent learning rate limit and random walk span respectively. All algorithms execute 10 independent runs on each instance and all experiments use the

TABLE VI
RESULTS OF HV METRIC FOR COMPARING SEVEN ALGORITHMS UNDER THE SAME MAXIMUM NUMBER OF EVALUATIONS

Ins	MOEA/D		NSGA-II		TSKEA		LRVMA		DQCE		MFLEDA		KTCE	
	mean	std	mean	std	mean	std	mean	std	mean	std	mean	std	mean	std
10J2F	0.0566-	0.012	0.0675-	0.025	0.2271=	0.013	0.2122-	0.015	0.2424=	0.020	0.0924-	0.016	0.2343	0.021
20J2F	0.0703-	0.017	0.0491-	0.012	0.2780=	0.006	0.1764-	0.019	0.2766=	0.013	0.1158-	0.016	0.2794	0.009
20J3F	0.0524-	0.014	0.0608-	0.011	0.2440-	0.006	0.2057-	0.024	0.3291-	0.020	0.1603-	0.017	0.3537	0.011
30J2F	0.0473-	0.007	0.0524-	0.009	0.2735-	0.003	0.1667-	0.016	0.3004-	0.010	0.1333-	0.006	0.3071	0.006
30J3F	0.0515-	0.011	0.0551-	0.005	0.2697-	0.004	0.1985-	0.008	0.3407=	0.008	0.1695-	0.015	0.3433	0.007
40J2F	0.0370-	0.008	0.0400-	0.008	0.2618-	0.004	0.1393-	0.021	0.2812=	0.013	0.1210-	0.008	0.2870	0.005
40J3F	0.0663-	0.014	0.0730-	0.013	0.2598-	0.003	0.1896-	0.021	0.3618=	0.013	0.2003-	0.008	0.3696	0.008
40J4F	0.0556-	0.012	0.0594-	0.009	0.2015-	0.003	0.1705-	0.028	0.3577=	0.016	0.1794-	0.014	0.3655	0.013
50J3F	0.0400-	0.007	0.0347-	0.003	0.2142-	0.004	0.1465-	0.020	0.3146=	0.010	0.1488-	0.009	0.3082	0.008
50J4F	0.0552-	0.009	0.0605-	0.012	0.2045-	0.004	0.1983-	0.009	0.3751-	0.014	0.1966-	0.013	0.3946	0.012
50J5F	0.0552-	0.010	0.0653-	0.006	0.1981-	0.005	0.2053-	0.043	0.3883-	0.012	0.2111-	0.015	0.4010	0.008
100J4F	0.0308-	0.005	0.0381-	0.005	0.0948-	0.004	0.1229-	0.020	0.3012-	0.007	0.1286-	0.008	0.3289	0.006
100J5F	0.0406-	0.008	0.0455-	0.007	0.0926-	0.002	0.1473-	0.043	0.3506-	0.010	0.1701-	0.008	0.3659	0.009
100J6F	0.0516-	0.007	0.0618-	0.006	0.0691-	0.004	0.1826-	0.022	0.3711-	0.010	0.1868-	0.010	0.4033	0.009
100J7F	0.0746-	0.016	0.0635-	0.007	0.0803-	0.002	0.1744-	0.043	0.4032-	0.009	0.2119-	0.011	0.4359	0.006
150J5F	0.0485-	0.005	0.0519-	0.009	0.1011-	0.002	0.1701-	0.018	0.3543-	0.009	0.1664-	0.005	0.3736	0.006
150J6F	0.0775-	0.013	0.0557-	0.013	0.0705-	0.003	0.1652-	0.040	0.3627-	0.005	0.1715-	0.007	0.3935	0.006
150J7F	0.0867-	0.022	0.0775-	0.014	0.0663-	0.002	0.2009-	0.025	0.3881-	0.008	0.1947-	0.006	0.4173	0.007
200J6F	0.0603-	0.015	0.0573-	0.003	0.0789-	0.003	0.1740-	0.009	0.3681-	0.007	0.1713-	0.005	0.4052	0.006
200J7F	0.0646-	0.014	0.0617-	0.004	0.0620-	0.001	0.2103-	0.010	0.3918-	0.007	0.1852-	0.010	0.4215	0.003
-/=/+	20/0/0		20/0/0		18/2/0		20/0/0		13/7/0		20/0/0			

TABLE VII
RESULTS OF IGD METRIC FOR COMPARING SEVEN ALGORITHMS UNDER THE SAME MAXIMUM NUMBER OF EVALUATIONS

Ins	MOEA/D		NSGA-II		TSKEA		LRVMA		DQCE		MFLEDA		KTCE	
	mean	std	mean	std	mean	std	mean	std	mean	std	mean	std	mean	std
10J2F	0.8030-	0.149	0.8466-	0.056	0.1301=	0.030	0.1560-	0.045	0.0984=	0.041	0.6783-	0.090	0.1107	0.053
20J2F	0.9773-	0.078	0.8963-	0.099	0.0554=	0.014	0.3121-	0.045	0.0564=	0.029	0.6402-	0.067	0.0467	0.020
20J3F	0.9605-	0.061	0.9607-	0.037	0.2998-	0.019	0.4125-	0.073	0.0793-	0.033	0.5544-	0.065	0.0399	0.018
30J2F	1.0050-	0.045	1.0283-	0.027	0.1187-	0.005	0.4704-	0.041	0.0518-	0.024	0.5981-	0.028	0.0293	0.009
30J3F	0.9957-	0.026	0.9561-	0.044	0.2268-	0.013	0.4369-	0.030	0.0428=	0.015	0.5195-	0.061	0.0339	0.008
40J2F	1.0888-	0.059	1.0900-	0.055	0.1165-	0.007	0.5182-	0.049	0.0593-	0.033	0.6198-	0.034	0.0392	0.010
40J3F	0.9224-	0.043	0.9482-	0.044	0.2993-	0.010	0.4490-	0.039	0.0475=	0.028	0.4734-	0.029	0.0311	0.016
40J4F	1.0146-	0.028	1.0117-	0.034	0.4739-	0.009	0.4891-	0.046	0.0596-	0.034	0.5496-	0.048	0.0318	0.016
50J3F	1.0973-	0.017	1.0911-	0.055	0.3501-	0.012	0.5356-	0.029	0.0422=	0.024	0.5616-	0.037	0.0529	0.017
50J4F	0.9731-	0.026	0.9895-	0.032	0.4658-	0.012	0.5034-	0.023	0.0875-	0.031	0.5079-	0.042	0.0500	0.019
50J5F	0.9645-	0.029	1.0029-	0.009	0.5416-	0.015	0.4240-	0.034	0.0499-	0.020	0.4736-	0.041	0.0346	0.009
100J4F	1.1110-	0.035	1.1348-	0.026	0.8131-	0.018	0.6342-	0.023	0.0911-	0.018	0.6545-	0.037	0.0168	0.014
100J5F	1.0328-	0.023	1.0811-	0.026	0.8492-	0.008	0.5561-	0.041	0.0677-	0.026	0.5678-	0.029	0.0236	0.015
100J6F	0.9744-	0.010	0.9778-	0.045	0.9211-	0.016	0.5513-	0.036	0.1010-	0.022	0.5664-	0.034	0.0265	0.013
100J7F	0.8868-	0.070	0.8530-	0.021	0.9008-	0.005	0.4877-	0.045	0.0803-	0.022	0.5377-	0.033	0.0160	0.010
150J5F	1.0222-	0.035	1.0130-	0.018	0.8172-	0.009	0.5392-	0.016	0.0754-	0.022	0.6017-	0.016	0.0220	0.015
150J6F	1.0135-	0.064	0.9337-	0.029	0.9234-	0.009	0.5390-	0.033	0.0954-	0.012	0.6107-	0.023	0.0275	0.009
150J7F	0.9153-	0.066	0.8653-	0.021	0.9428-	0.005	0.4939-	0.038	0.0777-	0.017	0.5725-	0.019	0.0185	0.010
200J6F	0.9772-	0.048	0.9075-	0.019	0.9021-	0.009	0.5471-	0.012	0.1021-	0.018	0.6279-	0.020	0.0153	0.010
200J7F	0.8947-	0.089	0.8426-	0.011	0.9590-	0.005	0.5128-	0.026	0.0805-	0.017	0.6073-	0.029	0.0107	0.005
-/=/+	20/0/0		20/0/0		18/2/0		20/0/0		15/5/0		20/0/0			

same termination condition, including: the maximum number of evaluations $MaxNFEs = 200 \times \sum_{i=1}^N n_i$ or the CPU time $CT_{max} = N \times (n_f - 1) (s)$.

1) *Comparison experiments under the same maximum number of evaluations:* Tables VI and VII respectively present the mean values and standard deviations of the seven algorithms on the HV and IGD metrics. Simultaneously, the concluding row present t-test ($\alpha = 0.05$) statistical significance results. “-” denotes that the comparison algorithm is markedly inferior to KTCE, “=” signifies that there is no significant difference exists and “+” means significantly superior to KTCE. In the experimental results, the optimal metric values are marked in bold. Additionally, Table VIII illustrates the ranking results and corresponding p -values derived from the Friedman rank-sum test, using a confidence level $\alpha = 0.05$.

The experimental analyses show that the KTCE is significantly better than MOEA/D, NSGA-II, TSKEA, LRVMA and MFLEDA. For DQCE, the KTCE outperforms it on most small-scale instances, but not significantly, and significantly

TABLE VIII
FRIEDMAN TEST RANKINGS ACROSS ALL ALGORITHMS UNDER THE SAME MAXIMUM NUMBER OF EVALUATIONS

MOEAs	HV		IGD	
	rank	p -value	rank	p -value
MOEA/D	6.30	4.385E-20	6.35	4.038E-20
NSGA-II	6.50		6.35	
TSKEA	4.15		4.25	
LRVMA	3.95		3.50	
DQCE	1.95		1.95	
MFLEDA	4.05		4.50	
KTCE	1.10		1.10	

outperforms it on large-scale instances. Therefore, the experimental results validate the superior efficacy of the KTCE for ES-DHFJSP.

For each algorithm, the solutions of 10 times running are integrated and sorted non-dominated to obtain the final pareto front which is shown in Fig. 5. The average running time of the KTCE and three advanced algorithms on all instances is

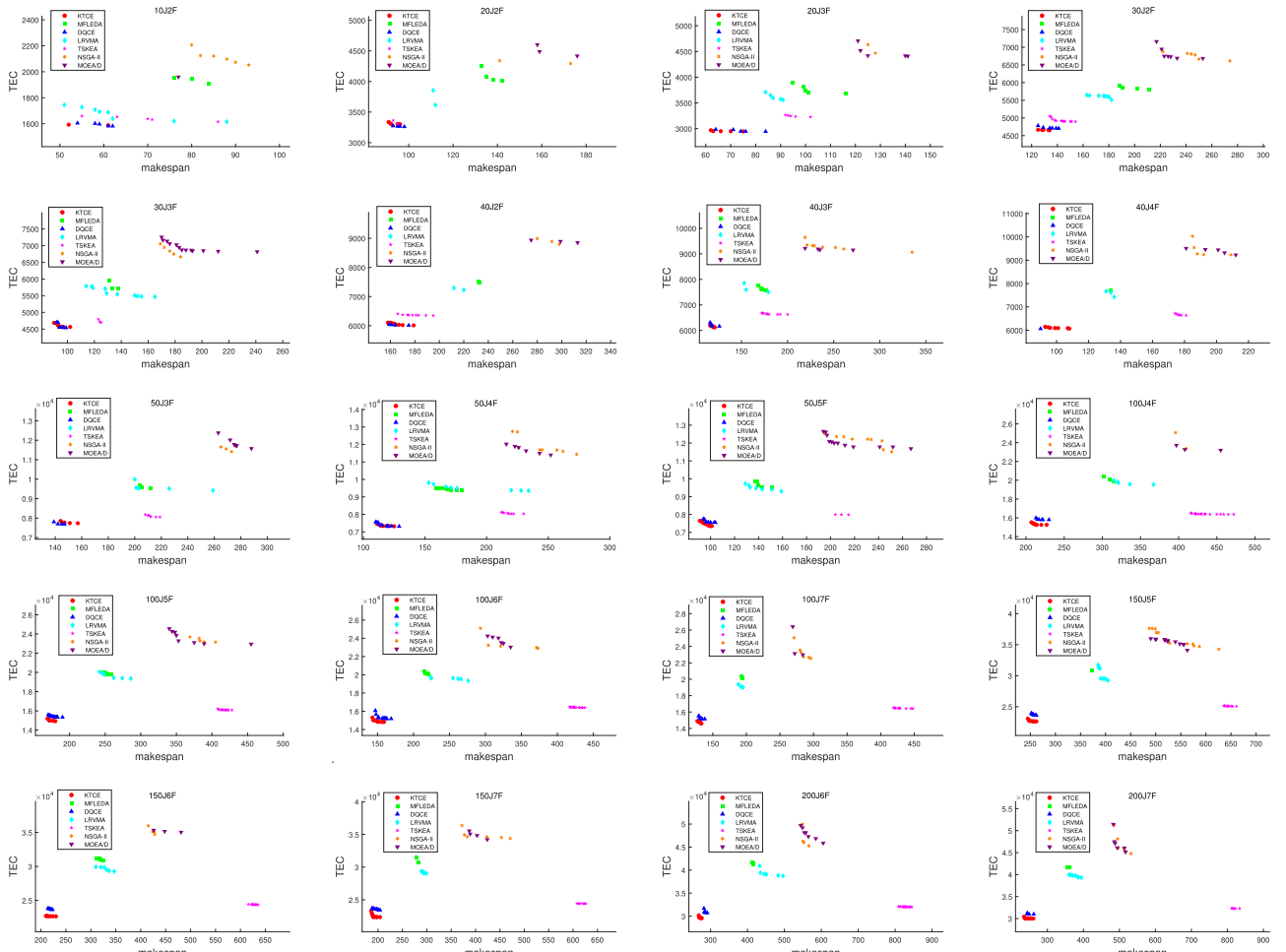


Fig. 5. Comparison of Pareto frontiers of different algorithms on each instance.

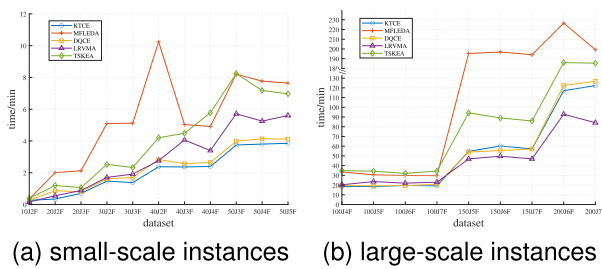


Fig. 6. The runtime analysis on different scale instances.

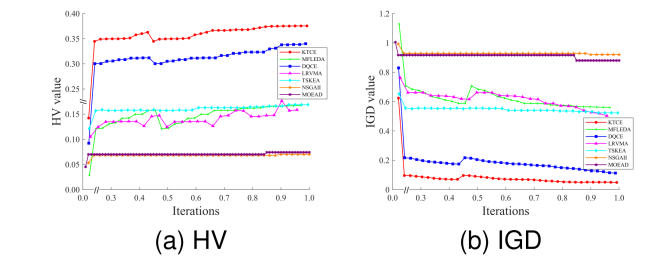


Fig. 7. The convergence behavior across seven algorithms on HV and IGD.

illustrated in Fig. 6. The results indicate that the KTCE reach the Pareto front with less time.

To facilitate a further efficiency comparison of the algorithms, the instance 100J4F is selected and optimized by seven optimization algorithms to analyze convergence. The curves of HV and IGD changing with the number of algorithm iterations are shown in Fig. 7. Due to the fact that different algorithms may complete different numbers of evaluations in one iteration, the proportion of completed evaluations in $MaxNFEs$ is utilized to show the changes in the two indicators.

Experimental results demonstrate that the greedy population initialization strategy significantly enhances both initial population quality and algorithmic convergence rate. Additionally, the factory allocation probability model-based knowledge transfer mechanism facilitates effective population evolution, further accelerating convergence performance. Moreover, the critical factory-based dynamic neighborhood search structure exhibits robust capabilities in exploring promising solution regions. These synergistic components enable KTCE to efficiently address the ES-DHFJSP.

TABLE IX
RESULTS OF HV METRIC FOR COMPARING SEVEN ALGORITHMS UNDER THE SAME CPU TIME

Ins	MOEA/D		NSGA-II		TSKEA		LRVMA		DQCE		MFLEDA		KTCE	
	mean	std	mean	std	mean	std	mean	std	mean	std	mean	std	mean	std
10J2F	0.1113-	0.021	0.1014-	0.016	0.2906+	0.011	0.3153+	0.022	0.2631+	0.022	0.0677-	0.020	0.2358	0.024
20J2F	0.1011-	0.011	0.1092-	0.016	0.3416+	0.003	0.2552-	0.020	0.2536=	0.035	0.0561-	0.017	0.2780	0.031
20J3F	0.0810-	0.016	0.0841-	0.023	0.3001-	0.008	0.2979-	0.017	0.3870=	0.016	0.0809-	0.018	0.3977	0.013
30J2F	0.0772-	0.011	0.0814-	0.012	0.3237+	0.005	0.2010-	0.017	0.3087=	0.020	0.0470-	0.007	0.3116	0.013
30J3F	0.0789-	0.011	0.0822-	0.014	0.3034-	0.005	0.2448-	0.030	0.3647-	0.015	0.0682-	0.016	0.3749	0.007
40J2F	0.0824-	0.009	0.0788-	0.010	0.3229=	0.005	0.1800-	0.021	0.2856-	0.013	0.0401-	0.006	0.3081	0.011
40J3F	0.0723-	0.009	0.0723-	0.009	0.2550-	0.005	0.2210-	0.014	0.3439-	0.011	0.0565-	0.009	0.3537	0.011
40J4F	0.0805-	0.007	0.0808-	0.010	0.2496-	0.004	0.2707-	0.031	0.3929-	0.014	0.0900-	0.018	0.4193	0.006
50J3F	0.0695-	0.006	0.0667-	0.009	0.2523-	0.006	0.1981-	0.013	0.3189-	0.020	0.0495-	0.009	0.3336	0.008
50J4F	0.0620-	0.008	0.0620-	0.007	0.2291-	0.004	0.2177-	0.033	0.3580-	0.010	0.0769-	0.010	0.3944	0.005
50J5F	0.0669-	0.006	0.0668-	0.016	0.2053-	0.005	0.2651-	0.022	0.3808-	0.017	0.1009-	0.013	0.3979	0.011
100J4F	0.0611-	0.004	0.0651-	0.010	0.1441-	0.005	0.1525-	0.017	0.2847-	0.020	0.0476-	0.007	0.3538	0.003
100J5F	0.0549-	0.009	0.0569-	0.005	0.1232-	0.003	0.1790-	0.010	0.3078-	0.015	0.0548-	0.006	0.3635	0.013
100J6F	0.0569-	0.006	0.0559-	0.011	0.0767-	0.005	0.1821-	0.026	0.3321-	0.012	0.0761-	0.009	0.3978	0.005
100J7F	0.0694-	0.006	0.0861-	0.025	0.0814-	0.006	0.2461-	0.011	0.3753-	0.013	0.1123-	0.010	0.4404	0.007
150J5F	0.0744-	0.008	0.0752-	0.007	0.1002-	0.003	0.1613-	0.024	0.2971-	0.008	0.0550-	0.008	0.3343	0.011
150J6F	0.0754-	0.004	0.0989-	0.020	0.0935-	0.003	0.1973-	0.011	0.3205-	0.006	0.0668-	0.013	0.3931	0.015
150J7F	0.0591-	0.004	0.0775-	0.019	0.0609-	0.001	0.1750-	0.028	0.3145-	0.008	0.0653-	0.010	0.3809	0.011
200J6F	0.0792-	0.005	0.0949-	0.017	0.0817-	0.002	0.1662-	0.021	0.2726-	0.014	0.0575-	0.008	0.3679	0.009
200J7F	0.0672-	0.006	0.0920-	0.021	0.0614-	0.002	0.1546-	0.027	0.2872-	0.018	0.0836-	0.010	0.3537	0.026
-/+	20/0/0		20/0/0		16/1/3		19/0/1		16/3/1		20/0/0			

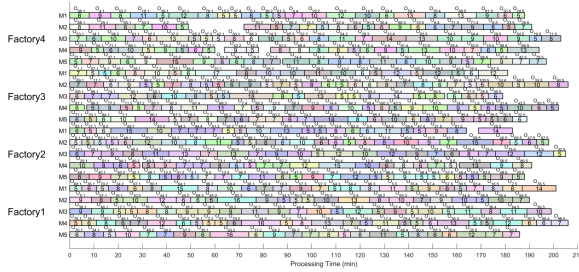


Fig. 8. The Gantt chart of the optimal schedule S_1 achieving minimum Makespan on instance 100J4F.

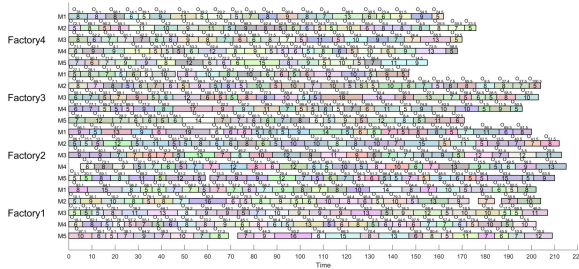


Fig. 9. The Gantt chart of the optimal schedule S_2 achieving minimum TEC on instance 100J4F.

The Gantt charts of the solution S_1 with the minimum makespan and the solution S_2 with the minimum TEC on 100J4F are shown in Figs. 8 and 9, respectively. The horizontal axis is the processing time, while the vertical dimension is the 5 machines in each factory. Each row shows the processing order of all processes on the machine. When the all processes on a machine are completed, then the machine stops running. The solution S_1 distributes all processes to each machine with optimal uniformity. For this reason, it can obtain the smaller makespan. The solution S_2 aims to minimize idle time between each process as much as possible to avoid machine idleness and

energy waste. Thereby, the results verify that KTCE can obtain the best scheduling solution.

2) *Comparison experiments under the same CPU time*: Due to the alteration in termination conditions, the relevant components of evaluation count are adapted within each algorithm. The current CPU time is defined as CT_c .

In KTCE, the current maximum number of neighborhood searches $maxLST$ is set to $\left\lceil \varepsilon \times \frac{CT_c}{CT_{max}} \right\rceil$. In MFLEDA, the learning rate $\beta = \beta_{max} - \frac{FEs}{maxFEs} \times (\beta_{max} - 0.01)$ is changed to $\beta = \beta_{max} - \frac{CT_c}{CT_{max}} \times (\beta_{max} - 0.01)$, where FEs is the number of function evaluations and $maxFEs$ is the maximum number of function evaluations. In TSKEA, the number of function evaluations of the first stage NFE_{s_1} is mathematically expressed as $NFE_{s_1} = MaxNFEs \times \frac{5}{7}$. Thus, in this experiment, $NFE_{s_1} = CT_{max} \times \frac{5}{7}$.

Tables IX and X present the experimental results of seven algorithms across 20 instances. Each algorithm's results are presented in two columns: mean values (mean) and standard deviations (std), while the best values are marked in bold. T-tests ($\alpha = 0.05$) are conducted to evaluate the statistical significance between each algorithm and KTCE, with the results displayed in the bottom row of each table. “-” denotes KTCE significantly outperforming the comparison algorithm. “=” signifies no statistically significant difference. “+” indicates the comparison algorithm achieves significantly better results than KTCE. Additionally, Table XI presents the Friedman rank-sum test results, including the ranks and corresponding p -values with the confidence level $\alpha = 0.05$.

The experimental results demonstrate that KTCE achieves an enhanced search efficiency. For TSKEA and LRVMA, the comparative analysis yields two key findings: (1) When CPU time serves as the termination criterion, they only outperform KTCE on the instances with $n_f = 2$, indicating significant limitations in solving complex problems; (2) When the maximum number of evaluations counts as the termination criterion, they consistently underperform KTCE across all instances. These

TABLE X
RESULTS OF IGD METRIC FOR COMPARING SEVEN ALGORITHMS UNDER THE SAME CPU TIME

Ins	MOEA/D		NSGA-II		TSKEA		LRVMA		DQCE		MFLEDA		KTCE	
	mean	std	mean	std	mean	std	mean	std	mean	std	mean	std	mean	std
10J2F	0.6928-	0.071	0.6923-	0.038	0.1430+	0.024	0.0731+	0.015	0.2080+	0.052	0.8803-	0.086	0.2645	0.052
20J2F	0.7798-	0.046	0.7252-	0.040	0.0176+	0.006	0.2339-	0.040	0.2448-	0.095	0.9649-	0.062	0.1687	0.086
20J3F	0.8970-	0.034	0.8820-	0.077	0.2412-	0.020	0.2503-	0.037	0.0490-	0.030	0.8770-	0.076	0.0269	0.023
30J2F	0.8849-	0.027	0.8704-	0.045	0.0523+	0.012	0.3834-	0.029	0.0961=	0.055	1.0556-	0.042	0.0859	0.033
30J3F	0.9042-	0.038	0.8828-	0.038	0.2180-	0.015	0.3187-	0.029	0.0577-	0.032	0.9359-	0.050	0.0348	0.014
40J2F	0.8445-	0.026	0.8461-	0.041	0.0246+	0.010	0.4038-	0.062	0.1135-	0.036	1.0864-	0.029	0.0655	0.028
40J3F	0.9504-	0.035	0.9463-	0.036	0.3148-	0.016	0.3987-	0.032	0.0824-	0.026	1.0151-	0.028	0.0497	0.023
40J4F	0.9019-	0.038	0.8843-	0.024	0.4028-	0.011	0.3121-	0.028	0.0788-	0.029	0.8668-	0.057	0.0190	0.006
50J3F	0.9207-	0.027	0.9408-	0.038	0.2922-	0.019	0.4135-	0.019	0.1010-	0.043	1.0300-	0.033	0.0551	0.016
50J4F	0.9573-	0.031	0.9672-	0.030	0.4262-	0.012	0.3966-	0.034	0.1031-	0.020	0.9048-	0.036	0.0261	0.008
50J5F	0.9669-	0.022	0.9553-	0.044	0.5224-	0.015	0.3263-	0.032	0.0739-	0.028	0.8144-	0.039	0.0345	0.016
100J4F	0.9714-	0.018	0.9139-	0.022	0.6643-	0.019	0.5572-	0.034	0.1842-	0.048	1.0482-	0.025	0.0185	0.004
100J5F	1.0119-	0.027	0.9668-	0.029	0.7476-	0.009	0.5223-	0.025	0.1774-	0.041	1.0294-	0.031	0.0380	0.003
100J6F	1.0191-	0.018	0.9233-	0.030	0.8993-	0.015	0.5073-	0.019	0.1659-	0.021	0.9336-	0.018	0.0171	0.008
100J7F	0.9636-	0.015	0.8245-	0.016	0.9008-	0.017	0.4274-	0.021	0.1581-	0.021	0.8211-	0.022	0.0204	0.009
150J5F	0.8901-	0.027	0.8693-	0.030	0.8060-	0.011	0.4949-	0.029	0.1441-	0.021	0.9864-	0.025	0.0366	0.021
150J6F	0.9443-	0.009	0.8129-	0.021	0.8687-	0.010	0.5268-	0.020	0.2179-	0.016	0.9682-	0.020	0.0423	0.034
150J7F	1.0026-	0.008	0.8497-	0.026	0.9513-	0.004	0.5322-	0.040	0.1880-	0.017	0.9675-	0.040	0.0336	0.021
200J6F	0.9155-	0.016	0.7662-	0.021	0.8929-	0.007	0.5495-	0.018	0.2758-	0.027	0.9963-	0.026	0.0336	0.019
200J7F	0.9614-	0.021	0.7776-	0.015	0.9564-	0.007	0.5607-	0.027	0.2357-	0.042	0.9036-	0.045	0.0517	0.033
-/+	20/0/0		20/0/0		16/0/4		19/0/1		18/1/1		20/0/0			

TABLE XI
FRIEDMAN TEST RANKINGS ACROSS ALL ALGORITHMS UNDER THE SAME CPU TIME

MOEAs	HV		IGD	
	rank	p-value	rank	p-value
MOEA/D	5.30	5.993E-18	5.15	1.135E-18
NSGA-II	6.00		6.25	
TSKEA	3.65		3.60	
LRVMA	3.25		3.20	
DQCE	2.25		2.25	
MFLEDA	6.25		6.25	
KTCE	1.30		1.30	

findings confirm limited local optima escaping performance in the three algorithms.

Comprehensive evaluations across both termination conditions confirm KTCE's superiority in two critical dimensions: accelerated convergence rate and enhanced local optimum avoidance capability.

VI. CONCLUSION

This paper develops a knowledge transfer-based co-evolutionary search algorithm for ES-DHFJSP. The algorithm has incorporated three key components: First, a greedy population initialization strategy is developed. FA vectors of one quarter of the initial population are generated based on average processing times, while the remaining individuals' FA vectors are generated stochastically. Implementation of this methodology elevates genotypic quality within the population and significantly enhances convergence rate. Second, a dynamic framework that has combined three critical factory-based neighborhood search strategies, enabling effective exploration of promising solutions while maintaining global search capabilities. Third, a knowledge transfer mechanism that has utilized factory allocation probability models to preserve high-quality factory assignment patterns and accelerate algorithmic convergence. Comprehensive experimental results across 20 benchmark instances have demonstrated

that KTCE has outperformed existing algorithms in addressing ES-DHFJSP.

Although the proposed KTCE exhibits excellent performance in ES-DHFJSP, it still has some deficiency that need to be solved in future work. Such as, in the later stage of iteration, the negative transfer may increase significantly and lead to falling into local optima. Additionally, the factors considered in the update of the probability model are also relatively limited. Therefore, investigating more efficient model to solve ES-DHFJSP problems is a future research direction.

REFERENCES

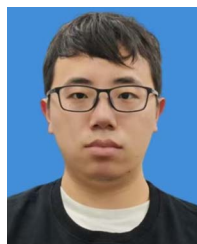
- [1] L. Deng, Y. Di, and L. Wang, "A reinforcement-learning-based 3-D estimation of distribution algorithm for fuzzy distributed hybrid flow-shop scheduling considering on-time-delivery," *IEEE Trans. Cybern.*, vol. 54, no. 2, pp. 1024–1036, Feb. 2024.
- [2] S. Cao, R. Li, W. Gong, and C. Lu, "Inverse model and adaptive neighborhood search based cooperative optimizer for energy-efficient distributed flexible job shop scheduling," *Swarm Evol. Computation*, vol. 83, 2023, Art. no. 101419.
- [3] Z. Zhang, Y. Fu, K. Gao, H. Zhang, and L. Wang, "A cooperative evolutionary algorithm with simulated annealing for integrated scheduling of distributed flexible job shops and distribution," *Swarm Evol. Computation*, vol. 85, 2024, Art. no. 101467.
- [4] H. Qin et al., "Energy-efficient iterative greedy algorithm for the distributed hybrid flow shop scheduling with blocking constraints," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 7, no. 5, pp. 1442–1457, Oct. 2023.
- [5] L. Meng, C. Zhang, Y. Ren, B. Zhang, and C. Lv, "Mixed-integer linear programming and constraint programming formulations for solving distributed flexible job shop scheduling problem," *Comput. Ind. Eng.*, vol. 142, 2020, Art. no. 106347.
- [6] L. De Giovanni and F. Pezzella, "An improved genetic algorithm for the distributed and flexible job-shop scheduling problem," *Eur. J. Oper. Res.*, vol. 200, no. 2, pp. 395–408, 2010.
- [7] M. Sheng, W. Ding, and W. Sheng, "Differential evolution with adaptive niching and reinitialisation for nonlinear equation systems," *Int. J. Syst. Sci.*, vol. 55, no. 10, pp. 2172–2186, 2024.
- [8] C. Wang, F. Qin, X. Xiang, H. Jiang, and X. Zhang, "A dual-population-based co-evolutionary algorithm for capacitated electric vehicle routing problems," *IEEE Trans. Transport. Electrification*, vol. 10, no. 2, pp. 2663–2676, Jun. 2024.
- [9] L. Wei, L. Jin, and X. Luo, "A robust coevolutionary neural-based optimization algorithm for constrained nonconvex optimization," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 6, pp. 7778–7791, Jun. 2024.

- [10] J. Chen et al., "A state-migration particle swarm optimizer for adaptive latent factor analysis of high-dimensional and incomplete data," *IEEE/CAA J. Automatica Sinica*, vol. 11, no. 11, pp. 2220–2235, Nov. 2024.
- [11] J. Fang, W. Liu, L. Chen, S. Lauria, A. Miron, and X. Liu, "A survey of algorithms, applications and trends for particle swarm optimization," *Int. J. Netw. Dyn. Intell.*, vol. 2, no. 1, pp. 24–50, Mar. 2023.
- [12] J. Cao, Y. Wang, Z. Bu, Y. Wang, H. Tao, and G. Zhu, "Compactness preserving community computation via a network generative process," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 6, no. 5, pp. 1044–1056, Oct. 2022.
- [13] G. Ma et al., "Estimating the state of health for lithium-ion batteries: A particle swarm optimization-assisted deep domain adaptation approach," *IEEE/CAA J. Automatica Sinica*, vol. 10, no. 7, pp. 1530–1543, Jul. 2023.
- [14] S. Hu, J. Lu, and S. Zhou, "Learning regression distribution: Information diffusion from template to search for visual object tracking," *Int. J. Netw. Dyn. Intell.*, vol. 3, no. 1, Mar. 2024, Art. no. 100006.
- [15] T. Chen, S. Li, Y. Qiao, and X. Luo, "A robust and efficient ensemble of diversified evolutionary computing algorithms for accurate robot calibration," *IEEE Trans. Instrum. Meas.*, vol. 73, 2024, Art. no. 7501814.
- [16] J. Xue and B. Shen, "A survey on sparrow search algorithms and their applications," *Int. J. Syst. Sci.*, vol. 55, no. 4, pp. 814–832, 2024.
- [17] C. Lu, L. Gao, J. Yi, and X. Li, "Energy-efficient scheduling of distributed flow shop with heterogeneous factories: A real-world case from automobile industry in China," *IEEE Trans. Ind. Inform.*, vol. 17, no. 10, pp. 6687–6696, Oct. 2021.
- [18] X. Yan, H. Zuo, C. Hu, W. Gong, and L. Gao, "Distributed heterogeneous flow shop scheduling method for dual-carbon goals," *IEEE Trans. Automat. Sci. Eng.*, vol. 22, pp. 7409–7420, 2025.
- [19] R. Li, W. Gong, L. Wang, C. Lu, Z. Pan, and X. Zhuang, "Double DQN-Based coevolution for green distributed heterogeneous hybrid flowshop scheduling with multiple priorities of jobs," *IEEE Trans. Automat. Sci. Eng.*, vol. 21, no. 4, pp. 6550–6562, Oct. 2024.
- [20] F. Ming, W. Gong, L. Wang, and Y. Jin, "Constrained multi-objective optimization with deep reinforcement learning assisted operator selection," *IEEE/CAA J. Automatica Sinica*, vol. 11, no. 4, pp. 919–931, Apr. 2024.
- [21] L. Hu, Z. Wang, H. Li, P. Wu, and N. Zeng, "l-DARTS: Light-weight differential architecture search with robustness enhancement strategy," *Knowl.-Based Syst.*, vol. 288, Mar. 2024, Art. no. 111466.
- [22] A. T. W. Min, Y. -S. Ong, A. Gupta, and C. -K. Goh, "Multiproblem surrogates: Transfer evolutionary multiobjective optimization of computationally expensive problems," *IEEE Trans. Evol. Comput.*, vol. 23, no. 1, pp. 15–28, Feb. 2019.
- [23] Y. Sang and J. Tan, "Intelligent factory many-objective distributed flexible job shop collaborative scheduling method," *Comput. Ind. Eng.*, vol. 164, 2022, Art. no. 107884.
- [24] X. Jin, X. Li, L. Gao, and L. Gui, "Distributed flexible job-shop scheduling problem based on hybrid chemical reaction optimization algorithm," *Complex Syst. Model. Simul.*, vol. 2, no. 2, pp. 156–173, 2022.
- [25] J. Li, X. Gu, Y. Zhang, and X. Zhou, "A hybrid genetic Tabu search algorithm for distributed flexible job shop scheduling problems," *J. Manuf. Syst.*, vol. 71, pp. 82–94, 2023.
- [26] F. Yu, C. Lu, J. Zhou, L. Yin, and K. Wang, "A knowledge-guided bi-population evolutionary algorithm for energy-efficient scheduling of distributed flexible job shop problem," *Eng. Appl. Artif. Intell.*, vol. 128, 2024, Art. no. 107458.
- [27] Y. Zhang and X. Gu, "A biogeography-based optimization algorithm with local search for large-scale heterogeneous distributed scheduling with multiple process plans," *Neurocomputing*, vol. 595, 2024, Art. no. 127897.
- [28] Z. Yang et al., "A Q-Learning-based improved multi-objective genetic algorithm for solving distributed heterogeneous assembly flexible job shop scheduling problems with transfers," *J. Manuf. Syst.*, vol. 79, pp. 398–418, 2025.
- [29] M. Yuan, S. Lu, L. Zheng, Q. Yu, F. Pei, and W. Gu, "Distributed heterogeneous flexible job-shop scheduling problem considering automated guided vehicle transportation via improved deep q network," *Swarm Evol. Computation*, vol. 94, 2025, Art. no. 101902.
- [30] G. Zhang, B. Liu, L. Wang, and K. Xing, "Distributed heterogeneous co-evolutionary algorithm for scheduling a multistage fine-manufacturing system with setup constraints," *IEEE Trans. Cybern.*, vol. 54, no. 3, pp. 1497–1510, Mar. 2024.
- [31] Q. Yan, H. Wang, and S. Yang, "A learning-assisted bi-population evolutionary algorithm for distributed flexible job-shop scheduling with maintenance decisions," *IEEE Trans. Evol. Comput.*, early access, May, 13, 2024, doi: 10.1109/TEVC.2024.3400043.
- [32] P. Fu, Z. Zhang, M. Huang, X. Guo, and L. Qi, "Multi-objective integrated energy-efficient scheduling of distributed flexible job shop and vehicle routing by knowledge-and-learning-based hyper-heuristics," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 9, no. 3, pp. 2137–2150, Jun. 2025.
- [33] Y. Chen, X. Liao, G. Chen, and Y. Hou, "Dynamic intelligent scheduling in low-carbon heterogeneous distributed flexible job shops with job insertions and transfers," *Sensors*, vol. 24, no. 7, 2024, Art. no. 2251.
- [34] F. Zhang, Y. Mei, S. Nguyen, and M. Zhang, "Multitask multiobjective genetic programming for automated scheduling heuristic learning in dynamic flexible job-shop scheduling," *IEEE Trans. Cybern.*, vol. 53, no. 7, pp. 4473–4486, Jul. 2023.
- [35] Q. Lin, Y. Ye, L. Ma, M. Jiang, and K. C. Tan, "Dynamic multiobjective evolutionary optimization via knowledge transfer and maintenance," *IEEE Trans. Syst., Man, Cybern. Syst.*, vol. 54, no. 2, pp. 936–949, Feb. 2024.
- [36] J. Zheng, L. Yu, Q. Song, J. Huang, G. Sheng, and Y. Chen, "Co-evolutionary fuzzy deep transfer learning for disaster relief demand forecasting," *IEEE Trans. Emerg. Topics Comput.*, vol. 10, no. 3, pp. 1361–1373, Jul./Sep. 2022.
- [37] M. Jiang, Z. Huang, L. Qiu, W. Huang, and G. G. Yen, "Transfer learning-based dynamic multiobjective optimization algorithms," *IEEE Trans. Evol. Comput.*, vol. 22, no. 4, pp. 501–514, Aug. 2018.
- [38] X. Wang, Z. Dong, L. Tang, and Q. Zhang, "Multiobjective multitask optimization-neighborhood as a bridge for knowledge transfer," *IEEE Trans. Evol. Comput.*, vol. 27, no. 1, pp. 155–169, Feb. 2023.
- [39] Y. Guo, G. Chen, M. Jiang, D. Gong, and J. Liang, "A knowledge guided transfer strategy for evolutionary dynamic multiobjective optimization," *IEEE Trans. Evol. Comput.*, vol. 27, no. 6, pp. 1750–1764, Dec. 2023.
- [40] G. Zhang, B. Liu, L. Wang, D. Yu, and K. Xing, "Distributed co-evolutionary memetic algorithm for distributed hybrid differentiation flow-shop scheduling problem," *IEEE Trans. Evol. Comput.*, vol. 26, no. 5, pp. 1043–1057, Oct. 2022.
- [41] H. Peng, Z. Luo, T. Fang, and Q. Zhang, "Micro many-objective evolutionary algorithm with knowledge transfer," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 9, no. 1, pp. 43–56, Feb. 2025.
- [42] H. Xu, A. K. Qin, and S. Xia, "Evolutionary multitask optimization with adaptive knowledge transfer," *IEEE Trans. Evol. Comput.*, vol. 26, no. 2, pp. 290–303, Apr. 2022.
- [43] C. Wang, J. Liu, K. Wu, and Z. Wu, "Solving multitask optimization problems with adaptive knowledge transfer via anomaly detection," *IEEE Trans. Evol. Comput.*, vol. 26, no. 2, pp. 304–318, Apr. 2022.
- [44] F. Zhang, Y. Mei, S. Nguyen, K. C. Tan, and M. Zhang, "Task relatedness-based multitask genetic programming for dynamic flexible job shop scheduling," *IEEE Trans. Evol. Comput.*, vol. 27, no. 6, pp. 1705–1719, Dec. 2023.
- [45] J. Fang et al., "A new particle swarm optimization algorithm for outlier detection: Industrial data clustering in wire arc additive manufacturing," *IEEE Trans. Automat. Sci. Eng.*, vol. 21, no. 2, pp. 1244–1257, Apr. 2024.
- [46] R. Li, W. Gong, and C. Lu, "A reinforcement learning based RMOEA/D for Bi-objective fuzzy flexible job shop scheduling," *Expert Syst. with Appl.*, vol. 203, 2022, Art. no. 117380.
- [47] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Trans. Evol. Comput.*, vol. 6, no. 2, pp. 182–197, Apr. 2002.
- [48] F. Zhao, Y. Du, C. Zhuang, L. Wang, and Y. Yu, "An iterative greedy algorithm for solving a multiobjective distributed assembly flexible job shop scheduling problem with fuzzy processing time," *IEEE Trans. Cybern.*, vol. 55, no. 5, pp. 2302–2315, May 2025.
- [49] R. Li, W. Gong, L. Wang, C. Lu, and X. Zhuang, "Surprisingly popular-based adaptive memetic algorithm for energy-efficient distributed flexible job shop scheduling," *IEEE Trans. Cybern.*, vol. 53, no. 12, pp. 8013–8023, Dec. 2023.
- [50] Z. Shao, W. Shao, and D. Pi, "LS-HH: A learning-based selection hyper-heuristic for distributed heterogeneous hybrid blocking flow-shop scheduling," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 7, no. 1, pp. 111–127, Feb. 2023.
- [51] W. Fang, B. Shen, A. Pan, L. Zou, and B. Song, "A cooperative stochastic configuration network based on differential evolutionary sparrow search algorithm for prediction," *Syst. Sci. Control Eng.*, vol. 12, no. 1, 2024, Art. no. 2314481.
- [52] C. Wang, Z. Wang, Q.-L. Han, F. Han, and H. Dong, "Novel leader-follower-based particle swarm optimizer inspired by multiagent systems: Algorithm, experiments, and applications," *IEEE Trans. Syst., Man, Cybern. Syst.*, vol. 53, no. 3, pp. 1322–1334, Mar. 2023.

- [53] R. C. Van Nostrand, "Design of experiments using the Taguchi approach: 16 steps to product and process improvement," *Technometrics*, vol. 44, no. 3, pp. 289–289, Aug. 2002.
- [54] Q. Zhang and H. Li, "MOEA/D: A multiobjective evolutionary algorithm based on decomposition," *IEEE Trans. Evol. Comput.*, vol. 11, no. 6, pp. 712–731, Dec. 2007.
- [55] R. Li, W. Gong, L. Wang, C. Lu, and S. Jiang, "Two-stage knowledge-driven evolutionary algorithm for distributed green flexible job shop scheduling with type-2 fuzzy processing time," *Swarm Evol. Computation*, vol. 74, 2022, Art. no. 101139.
- [56] R. Li, W. Gong, C. Lu, and L. Wang, "A learning-based memetic algorithm for energy-efficient flexible job-shop scheduling with Type-2 fuzzy processing time," *IEEE Trans. Evol. Comput.*, vol. 27, no. 3, pp. 610–620, Jun. 2023.
- [57] R. Li, W. Gong, L. Wang, C. Lu, and C. Dong, "Co-evolution with deep reinforcement learning for energy-aware distributed heterogeneous flexible job shop scheduling," *IEEE Trans. Syst., Man, Cybern. Syst.*, vol. 54, no. 1, pp. 201–211, Jan. 2024.
- [58] F. Zhao, M. Li, N. Zhu, and T. Xu and Jonrinaldi, "Multi-objective fitness landscape-based estimation of distribution algorithm for distributed heterogeneous flexible job shop scheduling problem," *Appl. Soft Comput.*, vol. 171, 2025, Art. no. 112780.



Tao Li received the M.S. degree in computer software and theory from Henan Normal University, Henan, China, in 2016, and the Ph.D. degree in software engineering from Harbin Engineering University, Harbin, China, in 2019. He is currently a Lecturer with the College of Computer and Information Engineering with Henan Normal University. His research interests include evolutionary computation and machine learning, and focusing on feature selection.



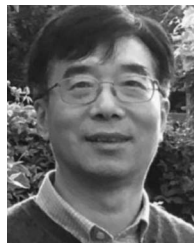
Xingchen Li is currently working toward the master's degree with the College of Computer and Information Engineering, Henan Normal University, Henan, China. His research interests include evolutionary computation and machine learning, and focusing on shop scheduling.



Junna Zhang (Member, IEEE) received the Ph.D. degree from the Institute of Network Technology, Beijing University of Posts and Telecommunications, Beijing, China, in 2019. She is currently a Professor with the College of Computer and Information Engineering, Henan Normal University, Henan, China. She is also a Vice President with the College of Computer and Information Engineering. She has authored or coauthored more than thirty papers in her research interests which include edge computing and service computing. She was the recipient of the Best Paper Award of CCF NCSC in 2022.



Dong Liu received the B.S. and M.S. degrees in computer science from Zhengzhou University, Zhengzhou, China, in 1998 and 2004, respectively, and the Ph.D. degree in computer science from Tianjin University, Tianjin, China, in 2013. He is currently a Professor with the College of Computer and Information Engineering, Henan Normal University, Xinxiang, China. His research interests include educational data mining and complex network analysis.



Zidong Wang (Fellow, IEEE) received the B.Sc. degree in mathematics from Suzhou University, Suzhou, China, in 1986, the M.Sc. degree in applied mathematics, and the Ph.D. degree in electrical engineering from the Nanjing University of Science and Technology, Nanjing, China, in 1990 and 1994, respectively. He is currently Professor of Dynamical Systems and Computing with the Department of Computer Science, Brunel University London, Uxbridge, U.K. From 1990 to 2002, he held teaching and research appointments with Universities in China, Germany, and U.K. He has authored or coauthored a number of papers in international journals. His research interests include dynamical systems, signal processing, bioinformatics, control theory, and applications. Dr. Wang is also a holder of the Alexander von Humboldt Research Fellowship of Germany, JSPS Research Fellowship of Japan, William Mong Visiting Research Fellowship of Hong Kong. He is (or was) the Editor-in-Chief of the *International Journal of Systems Science*, and *Neurocomputing*, and an Associate Editor for 12 international journals, including *IEEE TRANSACTIONS ON AUTOMATIC CONTROL*, *IEEE TRANSACTIONS ON CONTROL SYSTEMS TECHNOLOGY*, *IEEE TRANSACTIONS ON NEURAL NETWORKS*, *IEEE TRANSACTIONS ON SIGNAL PROCESSING*, and *IEEE TRANSACTIONS ON SYSTEMS, MAN, AND CYBERNETICS—PART C: APPLICATIONS AND REVIEWS*. He is also a member of the Academia Europaea, fellow of the Royal Statistical Society, and member of the Program Committee for many international conferences.